MODULE 11 · DOMAIN 2 - CONFIGURATION MANAGEMENT AND IAC

The AWS CDK and AWS SAM

Constructs and stacks, tagging and aspects, bootstrapping, context and logical IDs, testing, CDK Pipelines, and SAM's build, local and sync workflow.

DOMAIN 2 of 6 EXAM WEIGHT 17% MODULE 11 of 34

admin@sthithapragna.com

The CDK is a program that **writes** CloudFormation. SAM is a **shorthand** for it. Both end as a stack, so modules 9 and 10 still apply.

CDK or SAM?

AWS CDK

Infrastructure in a programming language

- TypeScript, Python, Java, C#, Go
- Any AWS service, any architecture
- Reusable constructs, loops, types, tests
- Best for large or shared infrastructure

AWS SAM

Serverless shorthand in YAML

- A transform over a CloudFormation template
- Functions, APIs, tables, state machines
- Local invoke and fast sync for developers
- Best for serverless applications

THE TIEBREAKER

Reusable components across many teams: CDK constructs. A serverless app with local testing: SAM. They can coexist.

Three levels of construct

LEVEL	WHAT IT IS	EXAMPLE
L1	One-to-one with a CloudFormation resource	CfnBucket: every property, no defaults
L2	An intent-based resource with sensible defaults	Bucket: encryption, grant methods, helpers
L3	A pattern combining several resources	A load-balanced Fargate service
Your own L3	Your organisation's approved pattern	A standard service with alarms and logging

ON THE EXAM

Reusable infrastructure patterns for many teams: custom L3 constructs, shared as a library.

Apps, stacks and environments

- An **app** contains **stacks**; each stack becomes one CloudFormation stack.
- One stack class, instantiated once per environment, with the differences passed in as **props**.
- Separate stacks per microservice let each deploy on its own schedule.
- Copying stack code per environment guarantees the environments drift apart.

> ON THE EXAM
>
> Consistent environments with small differences: one stack definition, parameterised, instantiated per environment.

One definition, three environments

```
1  const app = new cdk.App();
2
3  for (const env of ['dev', 'staging', 'prod']) {
4    new OrdersServiceStack(app, `orders-${env}`, {
5      env: { account: accounts[env], region: 'eu-west-2' },
6      stage: env,
7      minTasks: env === 'prod' ? 4 : 1,
8    });
9  }
10
11 cdk.Tags.of(app).add('cost-centre', 'orders');
12 cdk.Aspects.of(app).add(new AwsSolutionsChecks());
```

7 The only difference between environments, passed as a prop.

11-12 A tag and a compliance check applied to everything below the app.

ON THE EXAM

Tags.of(scope).add tags every taggable resource beneath that scope, in every stack.

Tags and aspects

- **Tags.of(scope).add** tags every taggable resource under that scope. Apply at the app for everything.
- An **aspect** visits every construct in a scope, to check or change it.
- **cdk-nag** is a set of aspects that fails synthesis on rule violations.
- Tagging resources one by one in each stack is the high-maintenance distractor.

> ON THE EXAM
>
> Same cost allocation tags across every microservice: Tags.of(app), once. Compliance checks at synth: an aspect.

From code to stack

01
Write
Constructs in your chosen language.

02
Synth
Templates and assets appear in cdk.out.

03
Diff
Compare with what is deployed.

04
Deploy
Assets to the bootstrap bucket and registry; stack updated.

05
Watch
cdk drift compares deployed stacks with reality.

ON THE EXAM

Synthesis produces ordinary CloudFormation. You can review it, test it, and scan it like any template.

Bootstrapping

- **cdk bootstrap** creates the CDKToolkit stack in each account and Region: a bucket, a registry, and roles.
- **--trust** lets a pipeline account deploy into this account.
- **--cloudformation-execution-policies** limits what deployments may create. By default it is administrator access.
- A custom permissions boundary can also be applied to roles the CDK creates.

> ON THE EXAM
>
> Restrict what CDK deployments can create in production: re-bootstrap with narrower execution policies.

Context and lookups

- Lookups such as **Vpc.fromLookup** query your account during synthesis.
- Results are cached in **cdk.context.json**. Commit that file.
- With the cache committed, synthesis is repeatable, and pipelines need no lookup access.
- Stacks without an explicit account and Region cannot use lookups.

> ON THE EXAM
>
> Synthesis gives different templates on different machines or days: commit cdk.context.json.

Logical IDs: the refactoring trap

- A resource's logical ID comes from its **path** in the construct tree.
- Moving or renaming a construct changes that path, so CloudFormation sees a **new** resource and **replaces** it.
- Keep the original ID with **overrideLogicalId**, or use **cdk refactor**, in preview since September 2025.
- **RemovalPolicy.RETAIN** keeps the old resource, but the new one still starts empty.

> ON THE EXAM
>
> Always run cdk diff after restructuring code. 'Replace' on a stateful resource means the logical ID changed.

Testing CDK code

- The **assertions** module checks synthesised templates: a resource exists, with these properties.
- **Snapshot tests** compare a whole template with an approved copy, catching unintended changes.
- Both run as ordinary unit tests in CodeBuild, before any deployment.
- cdk-nag aspects and Guard rules add policy checks on the same output.

> ON THE EXAM
>
> Prove every bucket the construct library creates is encrypted, before deploying: fine-grained assertions in unit tests.

CDK Pipelines, revisited

- A **synth** step builds the app; the pipeline then **updates itself** if its definition changed.
- Each **stage** is an environment; **waves** deploy several stages in parallel.
- **Pre** and **post** steps add tests or a manual approval around any stage.
- Target accounts must be bootstrapped with --trust for the pipeline account.

> ON THE EXAM
>
> It generates an ordinary CodePipeline pipeline, so everything from Domain 1 still applies underneath.

AWS SAM templates

- The **Transform: AWS::Serverless-2016-10-31** line tells CloudFormation to expand SAM resources.
- Resource types include Function, Api, HttpApi, SimpleTable, StateMachine and LayerVersion.
- **Policy templates** such as DynamoDBCrudPolicy grant scoped permissions in one line.
- **Connectors** declare that one resource talks to another, and SAM writes the permissions.

> ON THE EXAM
>
> Least-privilege Lambda permissions in SAM, with least effort: policy templates or connectors, not hand-written IAM.

The SAM CLI

COMMAND	WHAT IT DOES
`sam build`	Builds function code and dependencies
`sam local invoke, start-api`	Runs functions and APIs locally, in containers
`sam deploy --guided`	Deploys via CloudFormation; saves settings in samconfig.toml
`sam sync --watch`	Pushes code changes in seconds to a development stack
`sam logs, sam traces`	Reads CloudWatch logs and X-Ray traces
`sam pipeline init --bootstrap`	Generates a CI/CD pipeline and its resources

ON THE EXAM

sam local needs Docker. It tests your code, not IAM permissions or real service integrations.

sam deploy, or sam sync?

SAM SYNC

Speed, for development

- Updates function code directly where it can
- Seconds per change with --watch
- Can bypass a full CloudFormation deployment
- Development stacks only

SAM DEPLOY

Safety, for real environments

- Always a CloudFormation change set
- Minutes per change
- Traffic shifting and rollback apply
- Staging and production

THE TIEBREAKER

Faster developer iteration in the cloud: sam sync --watch. Never use it for production.

A CDK app uses Vpc.fromLookup to find an existing VPC. Developers synthesise locally, and a pipeline synthesises the same commit in CodeBuild.

The pipeline's CodeBuild role has no permission to read VPC details, so synthesis fails there. On developer machines, synthesis sometimes produces different templates from day to day.

What should the team do with the LEAST effort?

A Commit cdk.context.json to the repository so lookup results are cached and reused

B Grant the CodeBuild role full EC2 read access

C Replace every lookup with hard-coded VPC and subnet IDs throughout the code

D Run cdk deploy with the --force flag in the pipeline

◆ PAUSE AND DECIDE

What should the team do with the LEAST effort?

A Commit cdk.context.json

Lookups are cached in the repository. Synthesis is repeatable everywhere and needs no lookup access.

B EC2 read access

Fixes the pipeline error but not the changing templates. Synthesis still depends on the day.

C Hard-code IDs

Works, but throws away the lookup and spreads IDs through the code. More effort, more fragile.

D --force

Forces a deployment. Does nothing about how synthesis behaves.

THE SIGNAL

Two symptoms, one cause: lookups at synthesis time. The context file makes them deterministic.

> Production accounts were bootstrapped for the CDK with default settings. A security review notes that any CDK deployment there can create any resource, including IAM users.
>
> The security team wants CDK deployments in production limited to an approved set of managed policies, with the least operational effort and without changing developers' code.

What should the DevOps engineer do?

A. Write an SCP denying every action except those in the approved policies

B. Re-run cdk bootstrap in production with --cloudformation-execution-policies naming the approved policies

C. Delete the CDKToolkit stack from production accounts

D. Add a manual approval before every production deployment

◆ PAUSE AND DECIDE

What should the DevOps engineer do?

A SCP allow-list

Would restrict every principal in the account, not just CDK deployments. Far wider than asked.

B Re-bootstrap with execution policies

The execution role CloudFormation uses gets only the approved policies. Code unchanged.

C Delete the toolkit stack

Stops CDK deployments altogether.

D Manual approval

A person reviewing each change is not a limit on what can be created.

THE SIGNAL

Default bootstrap gives deployments administrator access. Execution policies at bootstrap are the fix.

A developer reorganises a CDK app, moving an existing DynamoDB table construct inside a new parent construct. The table holds production data.

Running cdk diff shows the table will be destroyed and a new table created. The team must keep the existing table and its data, with the least effort.

What should the developer do?

A Keep the table's original logical ID, for example with overrideLogicalId, then diff again

B Set RemovalPolicy.RETAIN on the table and deploy

C Export the data, deploy, then import the data into the new table

D Deploy with rollback disabled

◆ PAUSE AND DECIDE

What should the developer do?

A Keep the original logical ID

CloudFormation sees the same resource, so nothing is replaced. The diff confirms it.

B RemovalPolicy.RETAIN

The old table survives, orphaned. The application gets a new, empty table.

C Export and import

Works, with downtime and effort, to fix what is only a naming change.

D Rollback disabled

Changes what happens on failure. The replacement would still succeed.

THE SIGNAL

Code moved, diff says replace: the logical ID changed. Pin it back, or use cdk refactor now that it exists.

> Developers building a serverless app with SAM wait several minutes after every code change, while sam build and sam deploy update their personal development stacks.
>
> They want changes live in their development stacks within seconds, calling real AWS services. The production release process must not change.

What should the developers use?

A sam local invoke for every change, and no cloud deployments during development

B sam sync --watch against their development stacks

C Edit function code directly in the Lambda console

D Split the template into smaller stacks so each deployment is faster

◆ PAUSE AND DECIDE

What should the developers use?

A sam local only

Fast, but not real AWS services. Fails the stated requirement.

B sam sync --watch

Pushes code changes to the development stack in seconds. Production still uses deploy.

C Console edits

Drift from the template, lost on the next deployment, and no version control.

D Smaller stacks

Restructures the application to shave some minutes. Still not seconds.

THE SIGNAL

Seconds, in the cloud, development only: sam sync --watch. Production stays on sam deploy.

Module 11 in five lines

- CDK writes CloudFormation; SAM expands into it. Everything from modules 9 and 10 still applies.
- Custom L3 constructs for reuse; one stack class per environment via props; Tags.of(app) once.
- Bootstrap per account and Region; --trust for pipelines; execution policies to limit deployments.
- Commit cdk.context.json. Moving constructs changes logical IDs: diff, then pin or refactor.
- SAM: policy templates and connectors; sync for development speed, deploy for real environments.

> ON THE EXAM
>
> Next, module 12: Organizations and Control Tower. SCPs, RCPs, controls, Account Factory and customisations.